

EEE8089

M2M Technology IoT

Coursework, Part 2

Dr. Bystrov

School of Engineering
University of Newcastle upon Tyne

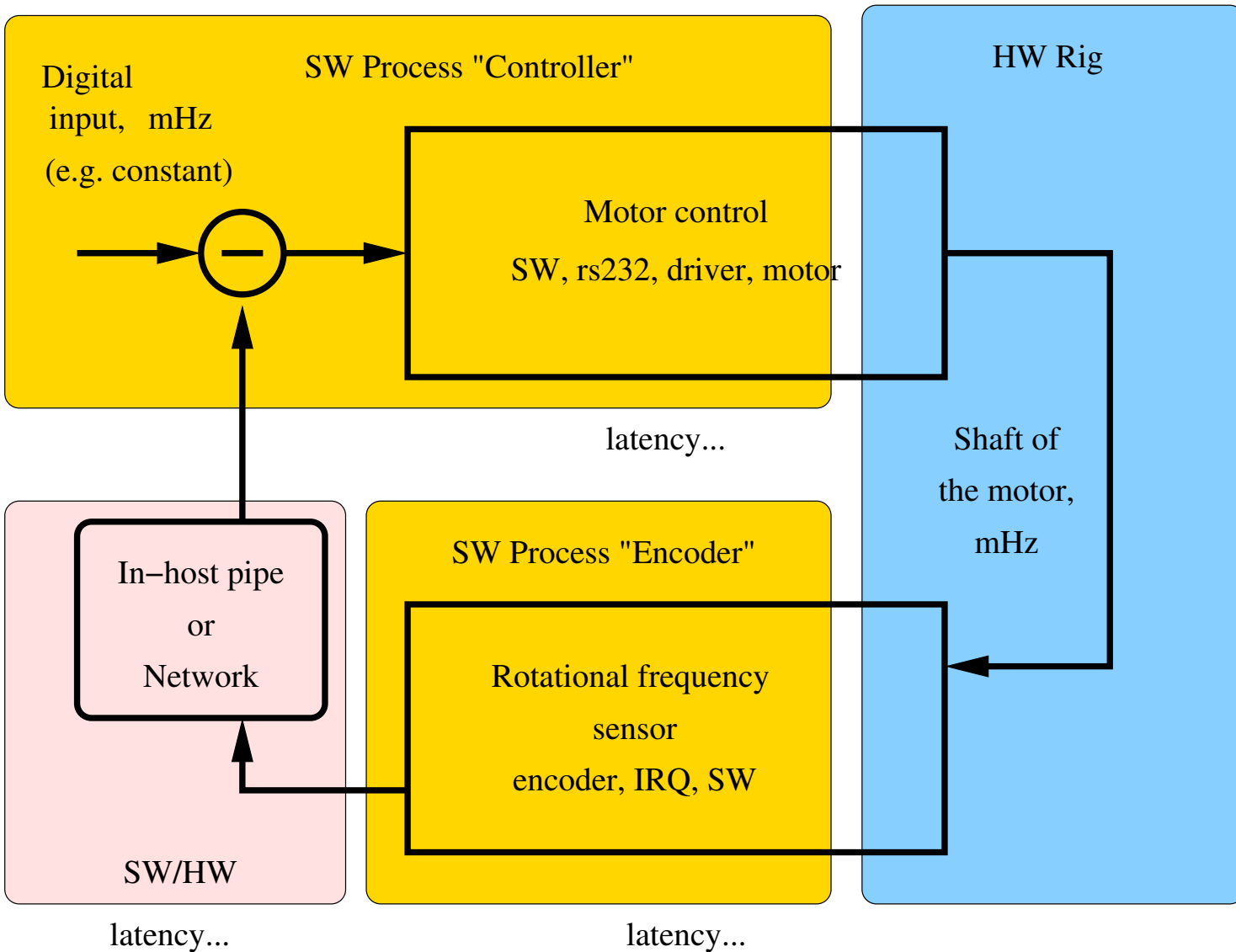
Aims

- (Part 1 in the previous lecture)
- Part 2 of the practical coursework
 - specifications
 - control system architecture
 - experiment 1
 - experiment 2
 - measuring the step response
 - tips on C-programming

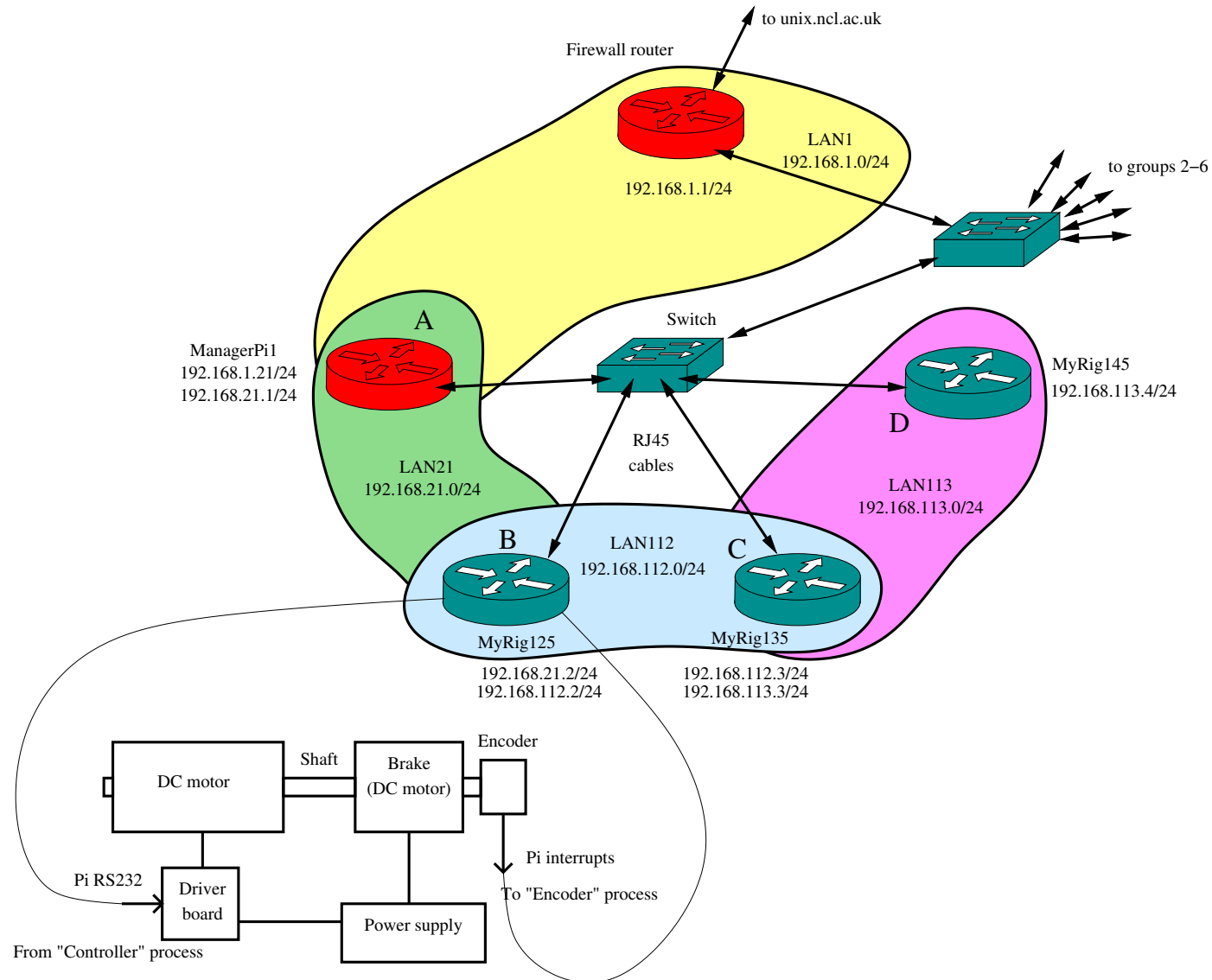
Part 2 Specs

- Implement a closed loop control system for an electrical motor
- Software components to implement:
 - Encoder program
 - choose the appropriate sample rate, format, method of frequency measurement
 - provide output at stdout stream
 - Controller program
 - target rotational speed 2Hz
 - Means for step response capture
 - plot motor speed vs time as a response to Heaviside step function
- Test stability on the same device and between two devices on the same or different LANs

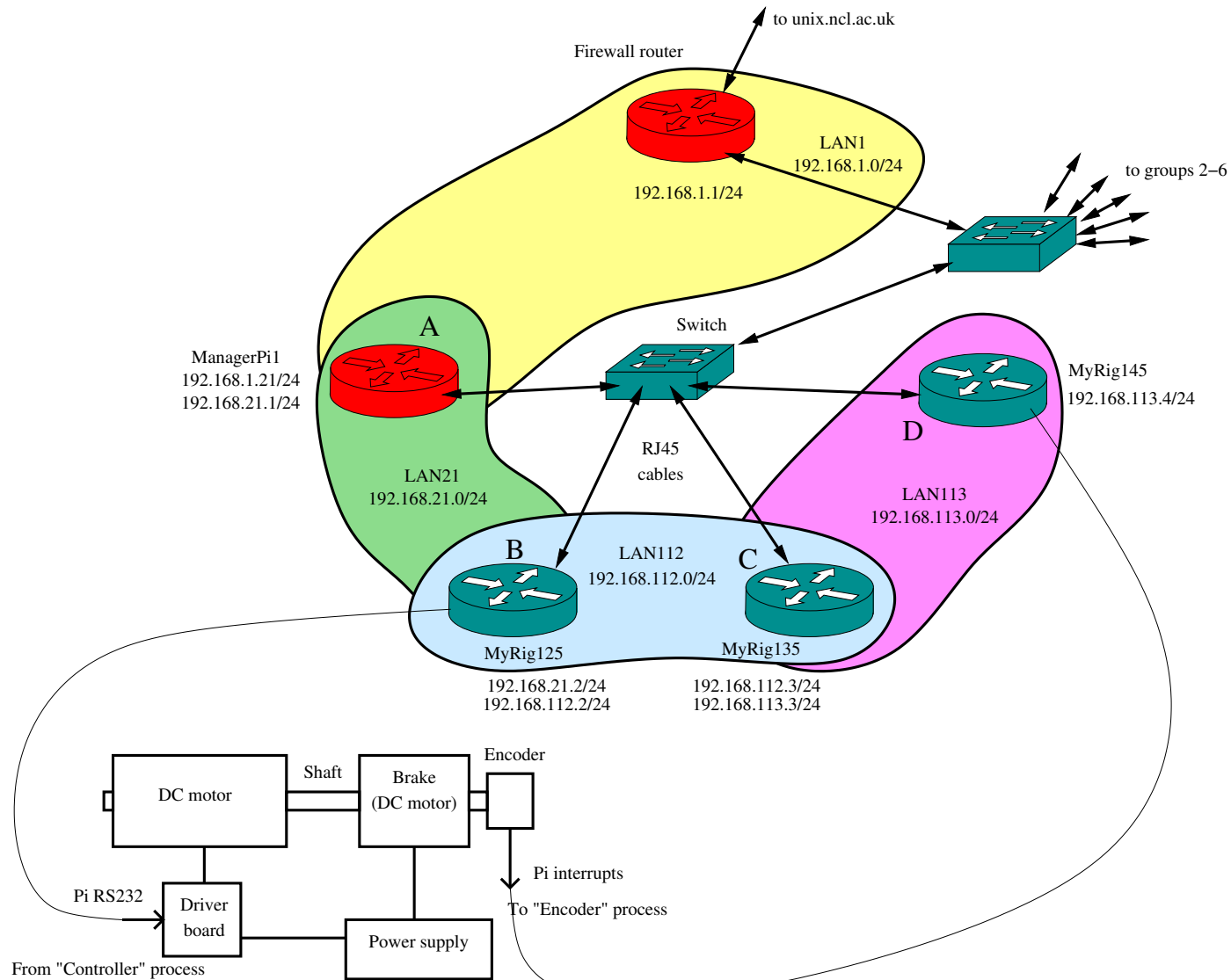
Control System Architecture



Connection to Pi-s, exp. 1



Connection to Pi-s, exp. 2



Step Response Measurements

- Modify Controller programme by replacing the constant input (2Hz) with a variable alternating between 0Hz and 2Hz with a sufficient period
- Record the output of Encoder in a file
 - use “tee” for sending a copy of stdout into a file
 - “man tee” – read the manual
- Find a way of plotting the data in this file, e.g.
 - start “gnuplot”, then type “plot 'samples.dat' with lines”
 - samples.dat: one data sample per line
 - install gnuplot: `sudo apt install gnuplot`

Stability issues

- If not stable - adjust the coefficients of the PID controller.
- Lower gain (coefficient values) usually results in a more stable system (and slower)

Programming: Libraries

```
#include <stdio.h> // printf() fflush()
#include <unistd.h> // usleep() lseek() read()
#include <stdlib.h> // system()
#include <poll.h> // poll()
#include <fcntl.h> // open()
// pthread_mutex_lock() pthread_mutex_unlock
#include <pthread.h>
// if you need a timer in case of interval measurement
#include <time.h>
```

Programming: setup GPIO

```
system("echo 9 > /sys/class/gpio/unexport;\n\necho 9 > /sys/class/gpio/export;\n\necho in > /sys/class/gpio/gpio9/direction;\n\necho rising > /sys/class/gpio/gpio9/edge;\n\necho 10 > /sys/class/gpio/unexport;\n\necho 10 > /sys/class/gpio/export;\n\necho in > /sys/class/gpio/gpio10/direction;\n\necho none > /sys/class/gpio/gpio10/edge");
```

You can do it in a more sophisticated way – check first if the appropriate GPIO is already enabled...

The same can be done with file access commands of C...

Programming: Interrupt

```
struct pollfd pfd0; //see "man poll"  
pfd0.events = POLLPRI | POLLERR; // enable events  
pfd0.fd = open("/sys/class/gpio/gpio9/value", O_RDONLY |  
O_NONBLOCK); // open file  
  
lseek(pfd0.fd, 0, SEEK_SET); read (pfd0.fd, str, 1); // empty  
the read buffer  
poll(&pfd0, 1, 1000); // this will wait for the next interrupt  
  
lseek(pfd0.fd, 0, SEEK_SET); read (pfd0.fd, str, 1); // empty  
the read buffer  
//poll again, consider a loop...
```

Programming: read GPIO pin

```
int fd1; // file descriptor
char str[4];
fd1 = open("/sys/class/gpio/gpio10/value", O_RDONLY |
O_NONBLOCK);

lseek(fd1, 0, SEEK_SET); // rewind to the beginning of the
file
read (fd1, str, 1); // read the value

//...seek-read again, etc.
```

Programming: Frequency Measurement

Two threads:

1. interrupt poll() thread increments a global variable
2. timer thread reads the global variable every 20-50ms and calculates the frequency of interrupt. The encoder provides 500 interrupts per rotation.

Programming: Period Measurement

One thread:

In a loop waits for the interrupt. Starts the timer before and reads after poll(). Very low latency, but the accuracy is low, because the interrupt latency is fluctuating 20-40us, occasionally more. With method some sort of averaging of samples is required.

Programming: Rotation Direction

If GPIO10==1 at the time of IRQ at GPIO9, then clockwise, else anticlockwise.

Programming: Controller

- Reads the samples from stdin, Encoder defines the sample rate.
- The error is a difference between the reference value and the reading coming from Encoder.
- Integrators/differentiators – don't forget dt defined by the sample rate.
- The driver board uses the code 737 as zero input. A valid range of inputs is 737 ± 600 .
- The recommended range of speeds is $\pm 10\text{Hz}$, above that the interrupts are not fast enough.